

LangChain Calculator Agent – Daily Learning Guide

This guide summarizes the setup, learning path, key concepts, and resources for building and understanding a simple LangChain Calculator Agent using Gemini.

1. Project Goal

Build a simple AI agent that can answer user questions and automatically use a calculator tool when needed.

Learn:

- LangChain fundamentals
- Tools
- Agents
- LLM integration
- Prompting
- Debugging workflows

2. Environment Setup

Step 1: Create project folder
`mkdir langchain-projects`

Step 2: Open VS Code
code .

Step 3: Create virtual environment
Windows:
`python -m venv .env`

Activate:
`.env\Scripts\activate`

Step 4: Install dependencies
`pip install langchain`
`pip install langchain-google-genai`
`pip install python-dotenv`

Step 5: Create .env file

`GOOGLE_API_KEY=YOUR_API_KEY`

Step 6: Verify installation
`pip show langchain`
`pip show langchain-google-genai`

3. Calculator Agent Code Flow

User Question
↓
Agent
↓
LLM decides whether a tool is needed
↓
Calculator Tool
↓
Tool Result
↓
Final Response

4. Key Concepts Learned

LLM:

The language model that reasons and generates responses.

Tool:

A Python function exposed to the agent.

Agent:

Decides when to call tools.

Prompt:

Instructions provided to the model.

Temperature:

Controls randomness.

0.0 = deterministic

1.0 = creative

Messages:

HumanMessage, AIMessage, SystemMessage.

5. Daily Learning Plan

Day 1:

- Environment setup
- Gemini API
- First LLM call

Day 2:

- Prompt templates
- Message types

Day 3:

- Tools
- Calculator agent

Day 4:

- Multiple tools

Day 5:

- Memory and chat history

Day 6:

- RAG basics

Day 7:

- Mini project combining all concepts

6. Common Errors and Fixes

Error: API key required

Fix: Set `GOOGLE_API_KEY` in `.env`

Error: model not found

Fix: Use a valid Gemini model name.

Error: temperature out of range

Fix: Use values between 0.0 and 2.0.

Error: ImportError

Fix: Check installed package versions.

7. Official Resources

LangChain Documentation:

<https://docs.langchain.com/>

LangChain Python API:

<https://python.langchain.com/>

LangSmith:

<https://smith.langchain.com/>

Google AI Studio:

<https://aistudio.google.com/>

LangGraph:

<https://langchain-ai.github.io/langgraph/>

8. Key Takeaways

- LangChain orchestrates LLM applications.
- Agents can choose and use tools automatically.
- Tools are regular Python functions decorated with `@tool`.
- Use low temperature for coding agents.
- Learn by building projects rather than reading docs only.
- Debug often and inspect model behavior carefully.
- Progression: LLMs → Prompts → Tools → Agents → Memory → RAG.